

**Neuro-Adaptive Network Flow Orchestrator: A Deep Reinforcement Learning
Approach for Proactive Congestion Control in Software-Defined Campus
Networks
(Internal)**

Dhruvin Hitesh Bhudia:169646

Supervisor: Mr. Tiberius Tabulu

An Informatics and Computer Science Project Proposal Submitted to the School of
Computing and Engineering Sciences in partial fulfillment of the requirements for the
award of a Degree in Bachelor of Science in Informatics and Computer Science

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

June 2026

Declaration and Approval

The student declares that this project proposal has not been submitted to Strathmore University or any other University for the award of a Degree in Bachelor of Science in Informatics and Computer Science or any other Degree. To the best of the student's knowledge and belief, the research proposal contains no material previously published or written by another person except where due reference is made in the research proposal itself.

Student Admission Number: 169646

Sign: _____

Date: 12/06/2026

Supervisor's Signature:

Sign: _____

Date: 12/06/2026

Acknowledgment

The researcher would like to extend heartfelt gratitude to the assigned supervisor, Mr. Tiberius Tabulu, for the unwavering support, technical guidance, and expert mentorship provided throughout the conceptualization and formulation of this research proposal.

Abstract

Campus networks at universities are experiencing a increasing traffic demands, largely driven by a growing reliance on digital libraries, online collaboration software, multimedia tools, and cloud-based educational platforms. Traditional routing protocols such as OSPF rely primarily on static or reactive path selection mechanisms, that only trigger after network bottlenecks have already formed. This delayed response leads to a drop in throughput, higher packet loss, elevated latency, and the sub-optimal use of existing network capacity.

This study proposes the development of the Neuro-Adaptive Network Flow Orchestrator, an DRL-driven traffic engineering framework that integrates Software-Defined Networking and Deep Reinforcement Learning to proactively mitigate congestion within campus network environments. The proposed system will utilize a centralized SDN controller to collect real-time network telemetry, including link utilization, queue occupancy, throughput, packet loss, and latency metrics. A DRL-based neuro-adaptive model will analyze network conditions and dynamically generate optimized routing decisions while maintaining routing stability.

The system will be implemented and evaluated within an emulated university network environment using Mininet, the Ryu SDN framework, and PyTorch. Performance will be assessed against OSPF-based routing using latency, throughput, packet delivery ratio, congestion occurrence frequency, and link utilization efficiency as evaluation metrics. The study seeks to provide a cost-effective, adaptive, and software-driven solution for improving network performance without requiring extensive hardware upgrades.

Table of Contents

Declaration and Approval.....	ii
Acknowledgment.....	iii
Abstract.....	iv
Table of Contents.....	v
List of Tables.....	ix
List of Abbreviations.....	x
Chapter One: Introduction.....	1
1.1 Background.....	1
1.2 Problem Statement.....	2
1.3 General Objective.....	3
1.3.1 Specific Objectives.....	3
1.3.2 Research Questions.....	4
1.4 Justification.....	4
1.5 Scope and Limitations.....	5
1.5.1 Scope of the Project.....	5
1.5.2 Limitations and Delimitations of the Project.....	6
Chapter Two: Literature Review.....	7
2.1 Introduction.....	7
2.2 Network Telemetry Parameters Contributing to Congestion in Software-Defined Networking.....	7
2.2.1 Core Telemetry Metrics in Software-Defined Networking Routing.....	7
2.2.1.1 Traffic Volume and Link Utilization.....	8
2.2.1.2 Queue Occupancy and Buffer Management.....	8
2.2.1.3 Latency and Packet Loss.....	8
2.3 Review of Standard Routing and Intelligent Network Management Solutions.....	9
2.3.1 Standard Routing Processes and Network Management Challenges.....	9
2.3.2 Constraints of Enterprise Intelligent Platforms.....	10
2.3.2.1 Cisco DNA Center.....	10
2.3.2.2 Juniper Mist AI.....	11
2.3.2.3 Arista CloudVision.....	13
2.3.3 Emerging Machine Learning and Software-Defined Networking Implementations.....	14

2.3.4 Routing Stability in Reinforcement Learning-Based Traffic Engineering	16
2.4 Gaps in Existing Solutions.....	18
2.5 Conceptual Framework.....	18
Chapter Three: Development Methodology.....	21
3.1 Introduction.....	21
3.2 Research Paradigm.....	21
3.2.1 Data Acquisition.....	21
3.2.2 Data Preprocessing.....	22
3.2.3 Model Training.....	22
3.2.4 Model Validation and Testing.....	24
3.3 Software Development Methodology.....	25
3.3.1 Requirements Elicitation.....	27
3.3.2 System Prototyping.....	27
3.3.3 Algorithmic Integration.....	27
3.3.4 Iterative Refinement.....	27
3.3.5 Interface Construction.....	28
3.3.6 System Implementation.....	28
3.3.7 Testing and Deployment.....	28
3.4 System Analysis and Design.....	28
3.4.1 Use Case Diagram.....	28
3.4.2 Class Diagram.....	29
3.4.3 Entity Relationship Diagram.....	29
3.4.4 Database Schema.....	29
3.4.5 System Architecture.....	30
3.4.6 Wireframes.....	31
3.5 System Development Tools and Techniques.....	31
3.5.1 Mininet Network Emulator.....	32
3.5.2 Ryu Software-Defined Networking Framework.....	32
3.5.3 PyTorch Deep Learning Library.....	32
3.5.4 Deep Reinforcement Learning.....	32
3.5.5 Neuro-Adaptive Lyapunov Bounding.....	33
3.5.6 RESTful API Integration.....	33
3.6 System Deliverables.....	33

3.6.1 System Documentation.....	33
3.6.2 Authentication Module.....	34
3.6.3 Telemetry Module.....	34
3.6.4 NANFO Cognitive Routing Module.....	34
3.6.4.1 The Actor-Critic Brain.....	34
3.6.4.2 The Neuro-Adaptive Filter.....	34
3.6.5 Interactive Administrator Dashboard Module.....	35
References.....	36
Appendix.....	40
Appendix A1: Time Schedule.....	40
Appendix A2: Turn-It-In Report.....	41

Table of Figures

<i>Figure 2.1. Cisco DNA Center Assurance Dashboard.....</i>	<i>11</i>
<i>Figure 2.2. Marvis conversational assistant interface.....</i>	<i>12</i>
<i>Figure 2.3. Arista CloudVision telemetry flow.....</i>	<i>13</i>
<i>Figure 2.4. Conceptual Framework Diagram.....</i>	<i>20</i>
<i>Figure 3.1. Hybrid Agile Development Lifecycle.....</i>	<i>26</i>

List of Tables

Table 2.1: Comparative Analysis of Existing Intelligent Network Management and Traffic Engineering Approaches.....	16
Table 3.1: Relational Database Schema Specifications.....	30
Table 3.2: Hardware and Software Emulation Environment Specifications.....	31

List of Abbreviations

2FA - Two-Factor Authentication

A3C - Asynchronous Advantage Actor-Critic

AI - Artificial Intelligence

API - Application Programming Interface

BGP - Border Gateway Protocol

DDPG - Deep Deterministic Policy Gradient

DQN - Deep Q-Network

DRL - Deep Reinforcement Learning

ERD - Entity Relationship Diagram

GNN - Graph Neural Network

ICT - Information and Communication Technology

LAN - Local Area Network

MDP - Markov Decision Process

ML - Machine Learning

NANFO - Neuro-Adaptive Network Flow Orchestrator

OSPF - Open Shortest Path First

PPO - Proximal Policy Optimization

QoS - Quality of Service

SDN - Software-Defined Networking

UML - Unified Modeling Language

List of Equations

Equation 3.1: Traffic Matrix Formulation.....	22
Equation 3.2: System State Differential Equation.....	23
Equation 3.3: Neural Network Approximation.....	23
Equation 3.4: Lyapunov Stability Function.....	24
Equation 3.5: Neuro-Adaptive Update Law.....	24

Chapter One: Introduction

1.1 Background

The rapid digital transformation of higher education across Sub-Saharan Africa has fundamentally altered the operational requirements of institutional local area networks. At Strathmore University, located in Nairobi, Kenya, the academic environment operates as a digitally intensive academic environment where students, faculty members, and administrative staff increasingly rely on concurrently access cloud-based learning management systems, multimedia research databases, and real-time collaboration tools. Modern higher education institutions fundamentally rely on Campus Area Networks (CANs) to interconnect diverse academic, administrative, and research facilities. Serving as the digital backbone of a university, a CAN facilitates high-speed communication, centralized resource sharing, and seamless access to enterprise applications across geographically proximate buildings. As educational delivery and research methodologies become increasingly digitized, the demand for robust, high-capacity CAN infrastructure has surged significantly (Cisco, 2024; ITU, 2023). In this context, the reliance on sophisticated technological infrastructure is critical for delivering an institution's academic mandate. Specialized environments, such as the School of Computing and Engineering Sciences and innovation hubs like @iLabAfrica, continuously advance data-intensive technological research. (@iLabAfrica Research and Innovation Centre, 2024). Consequently, these environments inherently demand an adaptive CAN architecture capable of sustaining high-volume data transfers without localized bottlenecks.

This rapid technological transition has yielded immense educational benefits but has also exposed operational limitations in the physical and logical network infrastructure. While historical investments in the Kenyan telecommunications sector have successfully laid high-capacity fiber optic backbones connecting major urban centers and universities, the localized distribution, orchestration, and management of that bandwidth within campus boundaries remain highly inefficient. The current network architecture at Strathmore University relies heavily on traditional, decentralized routing protocols such as Open Shortest Path First (OSPF) and the Border Gateway Protocol (BGP). These legacy systems operate on static, threshold-based heuristics and reactive engineering principles, meaning that the infrastructure only adjusts

routing parameters and shifts data flows after a severe bottleneck has already formed and packet loss is actively occurring; traditional routing protocols operate using reactive mechanisms that adapt only after network conditions have changed (Rusek et al., 2019). Consequently, the university faces chronic network congestion, high latency, and severe server bottlenecks.

In a broader socioeconomic context, addressing these connectivity issues through the standard industry practice of hardware over-provisioning which involves purchasing high-end, proprietary enterprise routers and expanding physical bandwidth capacity is financially prohibitive. Kenyan institutions face steep challenges regarding hardware procurement, including volatile currency exchange rates, high import tariffs on electronics, and restricted capital budgets dedicated to Information and Communication Technology. The government of Kenya has previously imposed taxes and import declaration fees on computing equipment, which directly impacts the ability of educational institutions to scale their physical hardware in response to growing traffic demands (Communications Authority of Kenya, 2024). Furthermore, the campus is occasionally subjected to national grid power fluctuations; during such outages, the network infrastructure lacks adaptive, intelligent power-saving protocols, leading to the rapid drainage of backup power supplies and severe energy wastage as switches run at maximum capacity regardless of actual traffic demand (Kenya Power and Lighting Company, 2024). Therefore, optimizing the existing infrastructure through frugal innovation and intelligent software orchestration has emerged as an urgent administrative and technical imperative.

1.2 Problem Statement

University campus networks are increasingly required to support large volumes of dynamic traffic generated by learning management systems, cloud services, video conferencing platforms, digital libraries, and collaborative applications. These traffic patterns fluctuate significantly throughout the day and frequently resulting in congestion, increased latency, packet loss, throughput degradation, and reduced quality of service (Rusek et al., 2019).

Conventional routing frameworks like Open Shortest Path First (OSPF) rely heavily on reactive congestion management and rigid path computations to make forwarding decisions. As a result, the network only adapts its routes after experiencing noticeable

performance drops. While enterprise-grade solutions such as Arista CloudVision, Juniper Mist AI, and Cisco DNA Center offer advanced predictive analytics and network automation, they typically impose strict vendor lock-in, significant licensing costs, and proprietary hardware mandates (Cisco Systems, 2024). These limitations make such platforms challenging to adopt for academic institutions operating with limited budgets.

Furthermore, existing intelligent network management solutions predominantly focus on monitoring, anomaly detection, and predictive maintenance, with limited emphasis on autonomous traffic engineering that continuously adapts routing decisions while maintaining network stability (Navarro-Alarcon et al., 2020).

Despite advancements in intelligent network management and Software-Defined Networking technologies, existing solutions remain limited in their ability to provide autonomous, proactive traffic engineering that continuously adapts routing decisions before congestion occurs while remaining cost-effective for resource-constrained university environments. Furthermore, many commercially available platforms depend on proprietary hardware and licensing models that restrict accessibility and scalability within higher education institutions. Consequently, there remains a need for an adaptive, software-driven framework capable of leveraging real-time network telemetry to predict congestion and dynamically optimize traffic flows while maximizing the utilization of existing network infrastructure. This study seeks to address this gap through the development and evaluation of a Deep Reinforcement Learning-based traffic engineering framework within an SDN environment.

1.3 General Objective

To design, develop, and validate a Deep Reinforcement Learning-based Neuro-Adaptive Network Flow Orchestrator for proactive traffic engineering and congestion mitigation in Software-Defined campus network environments.

1.3.1 Specific Objectives

- i. To analyze network telemetry parameters, including link utilization, latency, throughput, queue occupancy, and packet loss, that contribute to congestion within Software-Defined Network environments.
- ii. To review standard routing approaches and existing intelligent network

management solutions and their limitations in addressing network congestion.

- iii. To design and develop a Deep Reinforcement Learning-based neuro-adaptive traffic engineering model within the SDN architecture that autonomously executes preemptive traffic shaping to regulate packet transmission rates and strict traffic policing to enforce bandwidth allocation and Quality of Service (QoS) constraints.
- iv. To validate the performance of the developed Neuro-Adaptive Network Flow Orchestrator against conventional routing approaches using latency, throughput, packet delivery ratio, congestion occurrence frequency, and link utilization efficiency as performance metrics.

1.3.2 Research Questions

- i. Which network telemetry parameters contribute most significantly to congestion within Software-Defined Network environments?
- ii. What limitations exist in traditional routing approaches and existing intelligent network management solutions when addressing dynamic network congestion?
- iii. How can a Deep Reinforcement Learning-based neuro-adaptive traffic engineering model be designed and integrated within a Software-Defined Networking architecture for proactive congestion management?
- iv. How does the proposed Neuro-Adaptive Network Flow Orchestrator perform compared to traditional routing approaches in terms of latency, throughput, packet delivery ratio, congestion occurrence frequency, and link utilization efficiency?

1.4 Justification

The development of the Neuro-Adaptive Network Flow Orchestrator (NANFO) system will be highly justified from technical, economic, and operational perspectives. Technically, relying on reactive routing protocols in an era of cloud computing and heavy multimedia usage results in unacceptable latency and severe service degradation. By leveraging a Software-Defined Networking paradigm, the control plane will be physically decoupled from the data plane, granting a centralized software controller a comprehensive, global view of the network state. Integrating Deep Reinforcement Learning into the SDN controller may provide a mechanism for adaptive traffic management capable of responding to changing network conditions.

thereby transforming a static, unyielding network into an autonomous, self-healing system capable of intelligent load balancing.

From an academic perspective, the study contributes to the growing body of research on the application of Deep Reinforcement Learning within Software-Defined Networking environments. The findings may provide insights into the effectiveness of adaptive traffic engineering approaches for congestion mitigation in resource-constrained campus networks, thereby informing future research and practical deployments in higher education institutions.

Economically, the proposed project perfectly aligns with the principles of frugal innovation, which is highly relevant to the Kenyan higher education market. Frugal innovation focuses on reducing the complexity and cost of a service while retaining or enhancing core functionalities (Bariu, 2020). Instead of relying on capital-intensive hardware upgrades to alleviate bottlenecks, the NANFO approach will extract maximum utility from existing switches and cabling through intelligent software orchestration. This approach has the potential to reduce infrastructure upgrade requirements and associated operational costs for institutional networks, making high-performance connectivity accessible and sustainable despite severe budget constraints and prohibitive import taxes. Furthermore, the inclusion of adaptive power-saving protocols will directly address the unique infrastructural challenge of frequent power outages in the region, ensuring that critical academic services remain online longer during grid failures by dynamically shutting down redundant switch ports during periods of low utilization.

1.5 Scope and Limitations

1.5.1 Scope of the Project

This project will encompass the theoretical formulation, software development, and simulated deployment of the NANFO system. The core scope includes the strict architectural separation of the underlying network data plane from the centralized control function, enabling the SDN controller to calculate and enforce dynamic routing decisions within the emulated environment. Furthermore, it involves the design of a Deep Reinforcement Learning (DRL) agent equipped with routing stability mechanisms to drive these traffic engineering processes, alongside the development of a comprehensive administrator dashboard for real-time visualization.

To ensure rigorous evaluation without disrupting active campus infrastructure, the implementation will heavily rely on network emulation for testing. This will be strictly confined to a virtualized environment utilizing the Mininet emulator and the Ryu SDN framework to accurately replicate the complex topological conditions and traffic dynamics of a university campus network.

1.5.2 Limitations and Delimitations of the Project

The primary limitation of this project involves the inability to deploy and test the NANFO system on the live, physical production network of Strathmore University due to the inherent risks of causing operational downtime during the experimental training phases of the reinforcement learning agent. Consequently, the model's efficacy must be evaluated using synthetic traffic matrices and historical data logs generated within an emulated environment, which may not capture every unpredictable anomaly present in real-world physical transmission media. Furthermore, the intensive computational overhead required to train deep neural networks is inherently constrained by the processing capabilities of the available development hardware. To strategically mitigate these environmental and hardware limitations, deliberate delimitations have been established to maintain a highly focused and achievable research trajectory. To reduce simulation complexity, the study will strictly confine its scope to wired Ethernet routing and local area network flow orchestration, explicitly excluding the analysis of wireless spectrum sharing mechanisms, radio access network optimizations, and cellular 5G slicing. Additionally, to align with the constraints of the emulated testing environment, the project entirely excludes the development of custom physical hardware switches, relying instead on the standard OpenFlow protocol to communicate with virtual software switches. Finally, to ensure the reinforcement learning agent's training remains computationally feasible and conceptually targeted, security and intrusion detection mechanisms will fall completely outside the study's purview, dedicating all system resources and research efforts strictly to performance optimization, traffic engineering, and congestion mitigation.

Chapter Two: Literature Review

2.1 Introduction

The evolution of enterprise and academic network management has increasingly shifted toward automation, programmability, and artificial intelligence to cope with the explosive growth of connected devices and data-intensive applications. This chapter will review the state of academic network infrastructure in developing nations, specifically focusing on the severe challenges of hardware acquisition and maintenance in the Kenyan market. It will further evaluate the paradigm shift from traditional, decentralized routing to Software-Defined Networking and thoroughly critique recent scholarly applications that explore the integration of Machine Learning for traffic engineering. Finally, the review will identify the critical mathematical and practical gaps in algorithmic implementations, paving the way for the conceptual framework of the neuro-adaptive approach proposed in this study.

2.2 Network Telemetry Parameters Contributing to Congestion in Software-Defined Networking

Software-Defined Networking (SDN) has fundamentally reshaped traffic management by decoupling the control plane from the data plane, allowing for centralized, programmable routing. A cornerstone of this architecture is the ability to leverage protocols like OpenFlow and In-band Network Telemetry (INT) to collect real-time, hop-by-hop network status information directly through the data plane, subverting traditional methods that treat switches as inaccessible black boxes (Wang et al., 2023). To fully satisfy the objective of analyzing network telemetry parameters that contribute to congestion, it is critical to dissect the specific state-space inputsnamely traffic volume, link utilization, queue occupancy, latency, and packet loss. While Tang et al. (2024) emphasize link utilization as the primary congestion indicator, Zhou and Liu (2024) argue that queue occupancy provides earlier warning signals due to its direct relationship with buffer saturation. TWang et al. (2023) identify these metrics as critical indicators for congestion detection and routing optimization in SDN environments.

2.2.1 Core Telemetry Metrics in Software-Defined Networking Routing

In high-density environments like university campuses, traffic is rarely uniform. It is characterized by extreme volatility and sudden micro-bursts, heavily impacting the physical infrastructure's ability to maintain Quality of Service (QoS). SDN controllers must extract specific, granular telemetry data to feed into Deep Reinforcement Learning (DRL) algorithms to preempt congestion (Tang et al., 2024).

2.2.1.1 Traffic Volume and Link Utilization

Traffic volume refers to the absolute amount of data transiting a network, while link utilization measures the percentage of a network link's total bandwidth currently in use. During sudden campus micro-bursts such as thousands of students concurrently streaming multimedia at the end of a lecture traffic volume spikes abruptly. When utilizing traditional routing protocols, these bursts frequently converge on the shortest path links, driving link utilization to its absolute maximum capacity (Maximum Link Utilization, or MLU) and throttling overall throughput (Nguyen et al., 2024). As link utilization approaches 100%, the network's ability to absorb transient traffic spikes vanishes. Centralized SDN controllers monitor these two parameters in real-time to identify asymmetric loads, allowing the DRL agent to autonomously distribute flows across underutilized paths, thereby minimizing MLU across the topology (Tang et al., 2023).

2.2.1.2 Queue Occupancy and Buffer Management

Queue occupancy acts as a highly sensitive, preemptive indicator of impending network failure. When bursty traffic volume exceeds the active transmission rate of a switch port, excess packets are temporarily stored in hardware buffers. Sustained traffic bursts can lead to a severe condition known as bufferbloat, where long-lived flows persistently occupy limited buffer space, preventing new, short-lived requests from securing allocation (Zhou & Liu, 2024). Monitoring exact queue depth via SDN telemetry allows an intelligent orchestrator to detect congestion before the buffers physically overflow. This foresight enables the controller to preemptively shape traffic, regulate packet transmission rates, or enforce strict bandwidth priorities.

2.2.1.3 Latency and Packet Loss

If elevated queue occupancy is not proactively mitigated, hardware buffers inevitably

overflow, triggering unavoidable packet loss through mechanisms such as tail-drop. Packet loss acts as a destructive catalyst in IP networks; it forces the Transmission Control Protocol (TCP) to initiate rigid congestion avoidance algorithms and aggressively retransmit dropped packets. This cycle exponentially increases end-to-end latency and further saturates the already congested bottleneck links (Zhou & Liu, 2024). In a high-demand academic network, compounding latency severely degrades the performance of real-time applications. By treating high latency and packet loss as negative reward signals (penalties), neuro-adaptive reinforcement learning models iteratively train to calculate routing paths that circumvent congested nodes entirely, preserving semantic continuity across the network (Gamel et al., 2024).

2.3 Review of Standard Routing and Intelligent Network Management Solutions

To fully contextualize the necessity of the Neuro-Adaptive Network Flow Orchestrator (NANFO), it is critical to review both standard routing approaches and existing intelligent network management solutions. Examining these mechanisms highlights their inherent limitations in addressing dynamic, burst-driven network congestion, thereby justifying the transition to a predictive, DRL-based model.

2.3.1 Standard Routing Processes and Network Management Challenges

In traditional distributed networks, data transmission relies heavily on standard link-state routing processes such as Open Shortest Path First (OSPF). These standard routing protocols calculate the optimal path between nodes based on static link weights (e.g., link speed or administrative distance) and forward packets rigidly along those predetermined paths (Al-Sadi et al., 2023). While computationally inexpensive, these shortest-path-first methods fundamentally ignore real-time network load balancing. Their link options remain static; thus, they fail to adaptively adjust routing schemes when encountering sudden fluctuations in traffic demand.

To overcome the rigid nature of standard routing, enterprise architectures shifted toward intelligent network management paradigms, including Software-Defined Networking (SDN) and Software-Defined Perimeters (SDPs). SDN centralizes the control plane, theoretically allowing for dynamic, adaptive routing based on real-time

network states. However, existing adaptive management solutions still face profound challenges. Many current SDN controllers rely on reactive threshold-based heuristics they only calculate new path options *after* telemetry indicates that congestion thresholds (like high link utilization) have already been breached (Wang et al., 2023). Furthermore, integrating secure, identity-driven overlays like SDPs adds computational overhead, often exacerbating latency during the dynamic rerouting process (Nguyen et al., 2024). Consequently, the inability of legacy standard routing to react to dynamic bursts, coupled with the purely *reactive* (rather than predictive) nature of current SDN adaptive management, results in chronic congestion and sub-optimal utilization of existing network infrastructure.

2.3.2 Constraints of Enterprise Intelligent Platforms

2.3.2.1 Cisco DNA Center

Cisco DNA Center is a robust, intent-based network management system utilized globally by large enterprises and universities to automate provisioning and monitor network health (Cisco Systems, 2026). Smith et al. (2024) evaluated Cisco DNA Center in large campus environments and reported improvements in network visibility but highlighted high deployment costs. Network administrators use the application to design, provision, and apply policies across vast campus environments. It leverages artificial intelligence to provide predictive maintenance and machine reasoning workflows, analyzing telemetry data to pinpoint the root causes of connectivity issues (Cisco Systems, 2026).

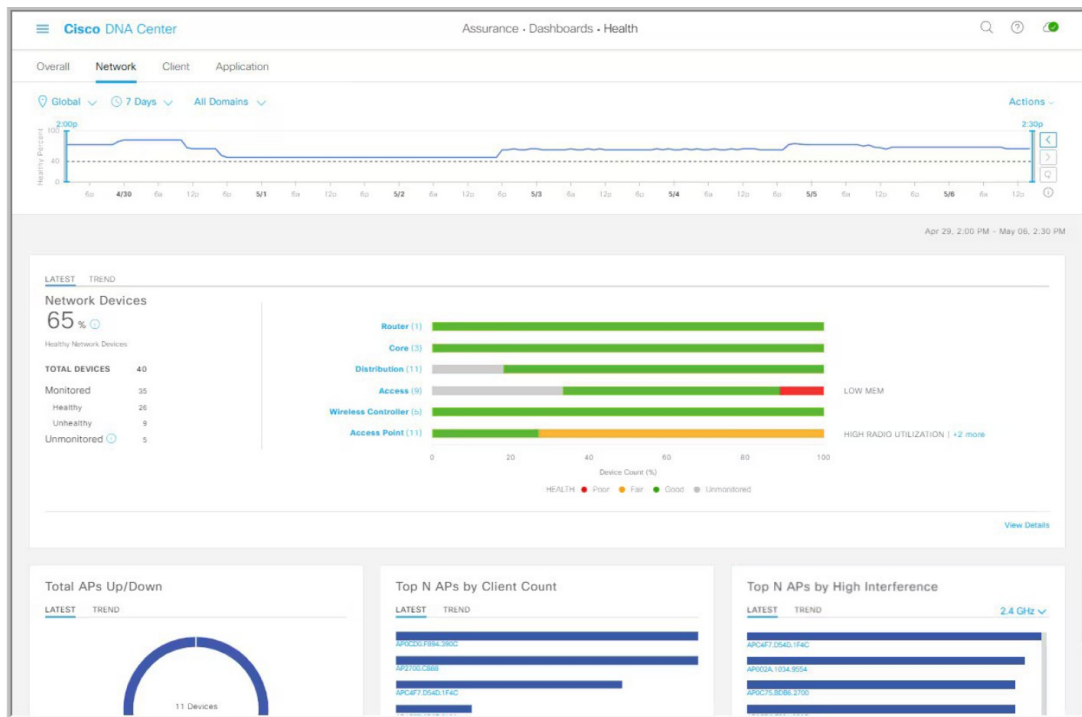


Figure 2.1. Cisco DNA Center Assurance Dashboard. From Cisco networking, cloud, and cybersecurity solutions, by Cisco Systems, 2026 (<https://www.cisco.com/>). Copyright 2026 by Cisco Systems, Inc.

Despite its advanced capabilities, Cisco DNA Center presents significant challenges in a resource-constrained context. The platform requires a steep learning curve and suffers from a highly complex graphical user interface (Cisco Systems, 2026). More critically, it enforces vendor-specific dependencies; the application is exclusively compatible with Cisco Catalyst hardware (Cisco Systems, 2026). For a university in a developing nation, this mandates exorbitant licensing costs and forces the institution to purchase expensive, proprietary hardware, completely contradicting the principles of frugal innovation.

2.3.2.2 Juniper Mist AI

Juniper Mist AI is a cloud-native platform that utilizes artificial intelligence and machine learning to optimize user experiences across wireless and wired local area networks (Juniper Networks, 2026). IT teams utilize the application's built-in conversational AI, known as Marvis, to perform natural language troubleshooting and proactive anomaly detection (Juniper Networks, 2026). The system is highly praised for its ability to dynamically adjust channel power and resolve performance issues automatically (Juniper Networks, 2026).

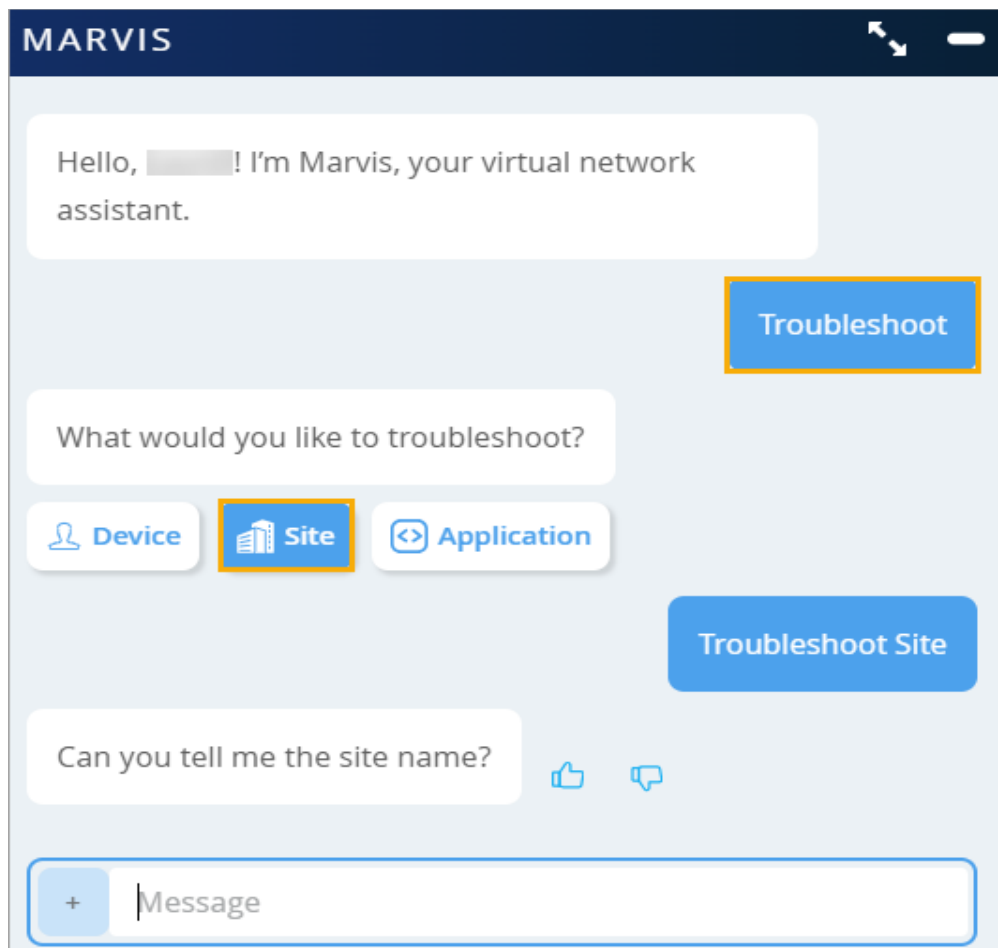


Figure 2.2. Marvis conversational assistant interface. From Marvis conversational assistant, by Juniper Networks, 2026 (<https://www.juniper.net/documentation/us/en/software/mist/mist-aiops/topics/concept/marvis-conversational-assistant.html>). Copyright 2026 by Juniper Networks, Inc.

While Juniper Mist AI excels in user experience optimization, its primary drawback lies in its cloud-dependency and subscription model (Juniper Networks, 2026). In environments like Kenyan universities where internet backhaul connections can be unstable, a purely cloud-managed orchestrator risks losing contact with the local area switches during service disruptions. Additionally, advanced features such as the Marvis virtual network assistant require premium add-on licenses, which severely strains limited institutional ICT budgets.

2.3.2.3 Arista CloudVision

Arista CloudVision is a network observability and automation platform primarily designed for data center and high-performance cloud environments (Arista Networks, 2026). Network engineers use CloudVision for zero-touch provisioning, real-time telemetry streaming, and automated change control across massive spine-leaf architectures (Arista Networks, 2026). It is widely adopted by hyperscale cloud providers due to its exceptional scalability and ultra-low latency handling (Arista Networks, 2026).

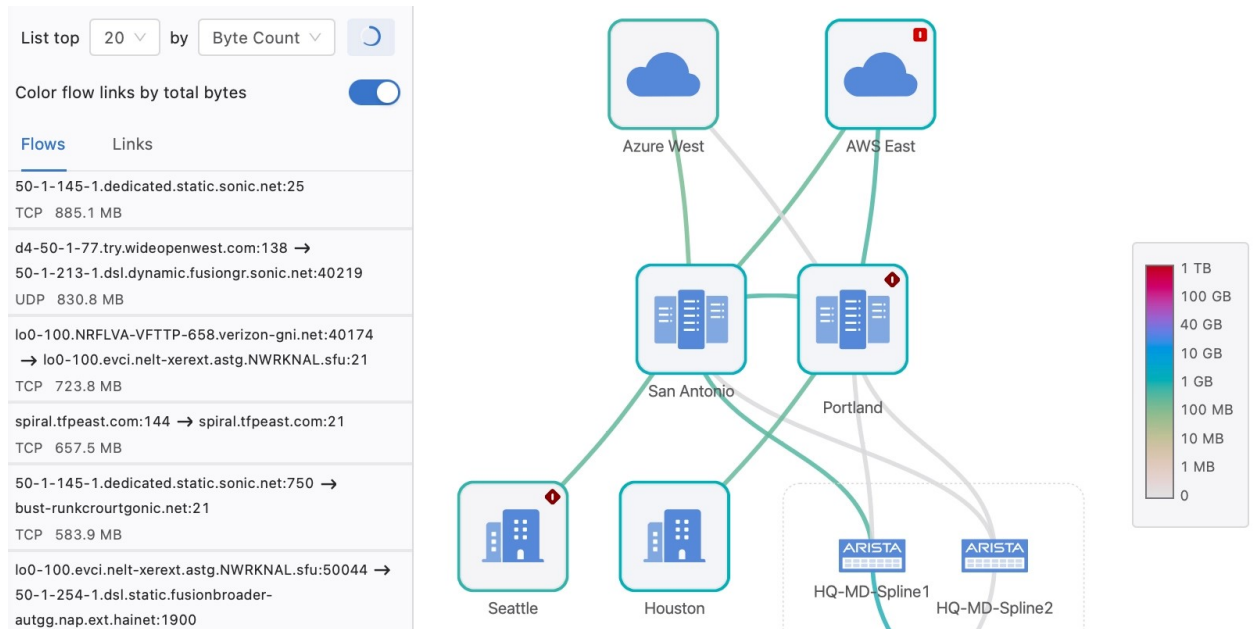


Figure 2.3. Arista CloudVision telemetry flow. From Arista CloudVision, by Arista Networks, 2026 (<https://www.arista.com/>). Copyright 2026 by Arista Networks, Inc.

The main disadvantage of Arista CloudVision is its explicit design focus on massive data centers rather than traditional, heterogeneous campus environments (Arista Networks, 2026). While it offers a simplified, single-binary operating system (EOS), its architecture is highly optimized for Arista's specific, high-cost switching hardware (Arista Networks, 2026). Furthermore, while it provides excellent automation, it lacks the deep, localized neuro-adaptive traffic engineering capabilities required to specifically manage unpredictable student traffic bursts on constrained, older equipment (Wong et al., 2024)

2.3.3 Emerging Machine Learning and Software-Defined Networking Implementations

The limitations of traditional shortest-path routing and reactive network management have motivated extensive research into the integration of Machine Learning (ML) and Artificial Intelligence (AI) within Software-Defined Networking (SDN) environments. Among these approaches, Deep Reinforcement Learning (DRL) has emerged as one of the most promising techniques for intelligent traffic engineering due to its ability to continuously learn optimal routing policies through interaction with dynamic network environments.

Several studies have demonstrated the effectiveness of DRL in improving network performance. Rusek et al. (2019) highlight that reinforcement learning-based traffic engineering frameworks can significantly improve throughput and reduce network congestion compared to traditional routing approaches. Similarly, Makris et al. (2024) report that DRL-based controllers are capable of adapting routing decisions in response to fluctuating traffic demands, thereby reducing latency and improving Quality of Service (QoS). Wang et al. (2024) further demonstrate that multi-agent reinforcement learning architectures outperform centralized rule-based routing mechanisms by achieving lower packet loss and more balanced resource utilization across large-scale network topologies.

Recent developments have also focused on improving the scalability and generalization capabilities of DRL-based routing systems. Almasan et al. (2022) propose the integration of Graph Neural Networks (GNNs) with reinforcement learning to capture topological relationships among network nodes more effectively. Building upon this work, Li et al. (2025) introduce Graph Transformer architectures that enable DRL agents to generalize routing policies across previously unseen network topologies without requiring extensive retraining. These advancements address one of the major limitations of conventional DRL systems, namely poor adaptability to changes in network structure.

Despite these advancements, DRL-based traffic engineering continues to face significant challenges associated with large state spaces and computational complexity. To address this limitation, Yang and Guo (2024) propose a contrastive learning-enhanced reinforcement learning framework that extracts compact traffic representations before policy optimization. Tang et al. (2023) validate a similar

approach using Deep Deterministic Policy Gradient (DDPG), demonstrating improved convergence rates and reduced Maximum Link Utilization (MLU) within SDN environments. However, these studies primarily focus on hybrid SDN architectures and do not fully address the requirements of centralized campus network orchestration.

Furthermore, several intelligent resource allocation frameworks have been proposed outside the routing domain. Janardhan et al. (2024) present an adaptive cloud resource management framework that utilizes predictive load balancing to reduce bottlenecks and improve service availability. Likewise, Gamel et al. (2026) introduce the Neuro-Adaptive Self-Healing (NASH) framework, which applies Hierarchical Reinforcement Learning (HRL) to maintain operational continuity during node failures. Although these approaches demonstrate the effectiveness of adaptive decision-making mechanisms, they focus primarily on application-level resource optimization and fault recovery rather than proactive traffic engineering within campus network infrastructures.

Collectively, the reviewed literature demonstrates that AI-driven networking solutions can substantially improve throughput, latency, and resource utilization. However, existing implementations predominantly target hybrid architectures, cloud resource management, fault tolerance, or topology generalization. Limited attention has been given to the development of cost-effective, stability-aware traffic engineering frameworks specifically designed for resource-constrained campus networks operating under highly dynamic traffic conditions.

Table 2.1: Comparative Analysis of Existing Intelligent Network Management and Traffic Engineering Approaches

Author(s)	Approach	Key Contribution	Limitation
Rusek et al. (2019)	Deep Reinforcement Learning for Traffic Engineering	Improved throughput and adaptive routing decisions	Scalability challenges in large state spaces
Almasan et al. (2022)	Graph Neural Network-based Optimization	Improved topology Routing awareness and routing efficiency	Limited focus on routing stability
Tang et al. (2023)	Contrastive Learning + DDPG	Reduced Maximum Link Utilization and faster convergence	Designed primarily for hybrid SDN environments
Yang & Guo (2024)	Contrastive Learning-assisted Reinforcement Learning	State-space reduction and improved policy optimization	Limited application in campus network environments
Wang et al. (2024)	Multi-Agent Reinforcement Learning	Reduced packet loss and improved load balancing	Increased computational complexity
Janardhan et al. (2024)	Predictive Resource Allocation	Cloud Effective bottleneck mitigation through load balancing	Operates at application layer rather than network routing layer
Li et al. (2025)	Graph Transformer-based Engineering	Generalization across unseen network topologies	Limited evaluation of routing stability
Gamel et al. (2026)	Neuro-Adaptive Healing Framework	Self- Fault recovery (NASH) resilience through HRL	Focuses on system recovery rather than traffic engineering

2.3.4 Routing Stability in Reinforcement Learning-Based Traffic Engineering

While recent advances in Deep Reinforcement Learning have demonstrated significant improvements in traffic engineering performance, routing stability remains one of the most critical challenges limiting the practical deployment of autonomous network optimization systems. In network environments, routing stability refers to the ability of a routing mechanism to maintain predictable and consistent forwarding

behaviour while adapting to changing traffic conditions. Excessive route fluctuations, commonly referred to as route oscillations, can generate additional control overhead, increase packet reordering, and negatively impact Quality of Service (QoS).

Conventional reinforcement learning algorithms continuously explore alternative actions to maximize long-term rewards. Although this exploration process is essential for learning optimal routing policies, it may introduce instability by causing frequent route changes in response to temporary fluctuations in network telemetry. Ding and Sun (2023) observe that highly adaptive reinforcement learning systems can experience policy oscillations when operating in environments characterized by rapidly changing traffic patterns. Such instability may reduce the overall effectiveness of otherwise high-performing traffic engineering solutions.

To address these challenges, researchers have increasingly explored the integration of control-theoretic stability principles with machine learning-based decision systems. Navarro-Alarcon et al. (2020) propose neuro-adaptive control mechanisms that incorporate stability constraints into learning-based systems, enabling adaptive behaviour while maintaining predictable system responses. Their findings suggest that combining adaptive learning with stability-aware control strategies can improve the reliability of autonomous decision-making systems operating in dynamic environments.

Within Software-Defined Networking environments, routing stability is particularly important because the centralized controller directly influences forwarding decisions across the entire network. Frequent policy updates may generate excessive control-plane overhead and introduce transient disruptions that offset the benefits of intelligent traffic engineering. Consequently, the integration of stability-aware neuro-adaptive mechanisms alongside Deep Reinforcement Learning presents a promising approach for balancing adaptability and operational consistency.

Therefore, while existing studies have successfully demonstrated the potential of reinforcement learning for traffic engineering, limited research has investigated the incorporation of explicit stability constraints within DRL-driven routing frameworks for campus network environments. This limitation represents a significant research gap and motivates the development of the Neuro-Adaptive Network Flow Orchestrator proposed in this study.

2.4 Gaps in Existing Solutions

The reviewed literature demonstrates significant advancements in network automation, telemetry analysis, and artificial intelligence-assisted network management. However, several critical research and implementation gaps remain evident.

First, many existing enterprise network orchestration platforms rely heavily on proprietary ecosystems, specialized hardware, and recurring licensing models, thereby limiting accessibility and affordability for institutions operating under constrained financial resources.

Second, although machine learning techniques are increasingly incorporated into network management systems, many implementations focus primarily on anomaly detection, predictive maintenance, and network monitoring rather than autonomous traffic engineering and routing optimization.

Third, existing studies on intelligent routing frequently emphasize performance improvement without adequately addressing routing stability during continuous adaptation. Limited research has explored the integration of Deep Reinforcement Learning with stability-constrained neuro-adaptive control mechanisms capable of maintaining predictable routing behaviour under dynamic network conditions (Navarro-Alarcon et al., 2020; Ding & Sun, 2023).

Finally, there is limited research focusing specifically on intelligent traffic engineering solutions tailored to university campus networks within developing regions where infrastructure expansion is constrained by financial and operational challenges.

Consequently, a gap exists for a cost-effective Software-Defined Networking framework that integrates Deep Reinforcement Learning and neuro-adaptive stability mechanisms to proactively mitigate congestion while maximizing the utilization of existing network infrastructure.

2.5 Conceptual Framework

The conceptual framework for this study is founded upon the integration of Software-Defined Networking and Deep Reinforcement Learning to facilitate intelligent traffic

engineering within campus network environments.

The framework begins with the collection of real-time network telemetry data from the data plane. These telemetry parameters include link utilization, queue occupancy, throughput measurements, packet loss statistics, and latency indicators. The collected information is transmitted to a centralized SDN controller, which maintains a global view of the network topology and current operational state.

The controller forwards the processed telemetry information to the Neuro-Adaptive Network Flow Orchestrator (NANFO) cognitive engine. Within this component, the Deep Reinforcement Learning agent continuously evaluates network conditions and generates routing decisions aimed at minimizing congestion, reducing latency, improving throughput, and optimizing resource utilization.

The generated routing policies are subsequently validated through a neuro-adaptive stability mechanism before being communicated back to the SDN controller. The controller then installs the optimized flow rules within the network switches, resulting in dynamic traffic redistribution across available paths.

The expected outcomes of the framework include improved network performance, enhanced congestion mitigation, reduced latency, improved bandwidth utilization, and more efficient use of existing networking infrastructure. The relationship among these variables is illustrated in Figure 2.4.

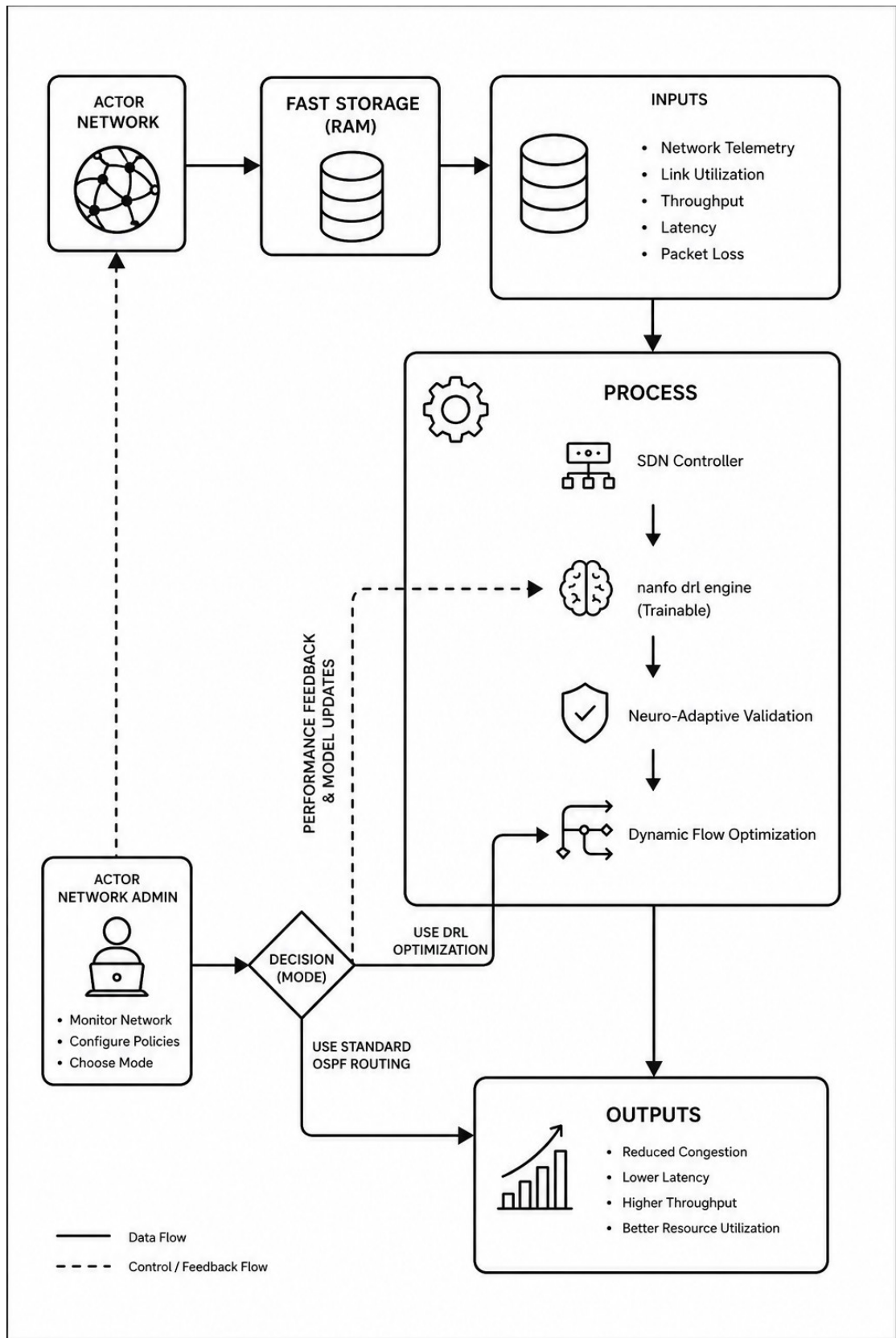


Figure 2.4. Conceptual Framework Diagram

Chapter Three: Development Methodology

3.1 Introduction

The successful realization of the Neuro-Adaptive Network Flow Orchestrator will require an exceptionally rigorous, scientifically grounded approach that seamlessly blends cutting-edge machine learning research with highly disciplined, practical software engineering. This chapter will meticulously detail the entire methodology that will be employed to build the NANFO system from the ground up. It will thoroughly delineate the research paradigm responsible for data handling and the highly complex mathematical training of the neuro-adaptive model. Furthermore, it will outline the hybrid Agile software development methodology used to carefully manage the project lifecycle, present the comprehensive system analysis and design artifacts, explicitly specify the development tools utilized, and define the final tangible system deliverables.

3.2 Research Paradigm

The research paradigm for this study will be fundamentally experimental and algorithmic, strictly following a structured pipeline of data acquisition, normalization preprocessing, mathematical model training, and rigorous validation within a controlled, sandboxed emulation environment.

3.2.1 Data Acquisition

The foundational basis of the reinforcement learning agent's cognitive capability will rely entirely on high-fidelity, real-time network telemetry. To establish a rigorous and accurate baseline that directly addresses the topological characteristics required for this study, the physical and logical network layout will be constructed using architectural diagrams and anonymized historical flow logs formally requested from the Strathmore University Information and Communication Technology (ICT) department. Data acquisition will then be executed dynamically within this meticulously replicated SDN environment. The Ryu controller will leverage the OpenFlow protocol to continuously poll the underlying virtual switches for state statistics at discrete, synchronized time intervals. The primary metrics extracted will include instantaneous port transmission rates, active queue lengths across all interfaces, packet loss ratios, and link latency measurements.

Furthermore, to accurately capture and analyze the complex, non-linear dynamics of campus data flows, the distributed traffic generators will not rely on purely artificial randomization. Instead, they will be deliberately seeded using the empirically acquired historical flow logs from the university. This hybrid, data-driven approach ensures that the synthetically generated traffic matrices accurately simulate the specific dense digital environment of Strathmore University, faithfully reproducing distinct academic cycles such as massive morning login spikes, sustained midday multimedia streaming, and sharp, aggressive post-lecture traffic bursts prior to their injection into the emulated topology.

3.2.2 Data Preprocessing

Raw network telemetry is inherently noisy, scaling disparately across different metrics. To ensure the mathematical stability of the neural network inputs, rigorous data preprocessing will be absolutely required. All extracted state variables will be subjected to Min-Max normalization to rigidly constrain values within a specific range. A critical preprocessing step will involve the continuous calculation of a normalized Traffic Matrix. To prevent the catastrophic gradient explosions that occur when neural networks process zero-valued denominators during ratio calculations, a stabilization constant must be explicitly added to the mathematical formulations. The normalized traffic matrix calculation will be mathematically defined as:

$$TM_{ij} = a + \frac{m_{ij} - \min(TM)}{\max(TM) - \min(TM)} \cdot (b - a) + 1e^{-6}$$

Equation 3.1: Traffic Matrix Formulation

Furthermore, to successfully provide the agent with deep spatial awareness, the normalized link states will be directly concatenated with the network adjacency matrix to form a comprehensive, multi-dimensional input state space suitable for processing by the integrated neural network layers (Makris et al., 2024).

3.2.3 Model Training

The core innovation of the NANFO system will lie in the mathematical formulation and training of its neuro-adaptive model. Actor-Critic reinforcement learning was selected because it effectively supports continuous action spaces and dynamic optimization problems commonly encountered in Software-Defined Networking environments. Furthermore, neuro-adaptive control techniques based on Lyapunov

stability theory have demonstrated effectiveness in maintaining bounded system behaviour during adaptive learning processes (Navarro-Alarcon et al., 2020; Ding & Sun, 2023). The network routing challenge will initially be formulated as a continuous-time Markov Decision Process defined by the tuple $(\mathbf{S}, \mathbf{A}, \mathbf{P}, \mathbf{R}, \mathbf{y})$. The state space $\mathbf{S}$ will represent the real-time normalized link utilizations and queue depths across all campus switches. The action space $\mathbf{A}$ will constitute a continuous vector of link-weight adjustments and flow split ratios assigned by the centralized controller. The reward function $\mathbf{R}$ will be meticulously engineered to heavily penalize high latency, punish packet loss exponentially, and simultaneously reward energy conservation.

To effectively solve this complex process while guaranteeing operational stability, the model will fuse an Actor-Critic neural network architecture with robust Neuro-Adaptive Control principles. Let the actual dynamic state of the network congestion be accurately described by the non-linear differential equation:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t)) + \mathbf{g}(\mathbf{x}(t))\mathbf{u}(t) + \mathbf{d}(t)$$

Equation 3.2: *System State Differential Equation*

Where $\mathbf{x}(t)$ represents the current network state vector, $\mathbf{u}(t)$ is the dynamic routing control input generated by the agent, $\mathbf{f}(\mathbf{x})$ and $\mathbf{g}(\mathbf{x})$ are completely unknown non-linear functions representing the complex intrinsic traffic behaviors of the student population, and $\mathbf{d}(t)$ represents massive external disturbances, such as sudden post-lecture traffic bursts. Because standard routing algorithms completely fail when $\mathbf{f}(\mathbf{x})$ is highly unpredictable, the NANFO system will employ a deep neural network to dynamically approximate these unknown dynamics online (Ding & Sun, 2023). The neural network approximation will be mathematically defined as:

$$\hat{\mathbf{f}}(\mathbf{x}) = \hat{\mathbf{W}}^T \sigma(\mathbf{V}^T \mathbf{x})$$

Equation 3.3: *Neural Network Approximation*

Where $\hat{\mathbf{W}}$ represents the rapidly adjusting output weight matrix of the neural network and $\sigma(\mathbf{V}^T \mathbf{x})$ denotes the specific sigmoid activation functions of the hidden layers (Navarro-Alarcon et al., 2020). To ensure that the learning process does not cause the routing configuration to become wildly unstable, the training will be strictly governed

by Lyapunov stability theory. The researcher will define a precise tracking error $e(t) = x(t) - x_d(t)$, where $x_d(t)$ is the desired optimal non-congested network state. A sliding surface $s(t) = \dot{e}(t) + \lambda e(t)$ will be defined to mathematically force the error dynamics to converge perfectly to zero (Navarro-Alarcon et al., 2020). The Lyapunov candidate function, which will act as an irrefutable mathematical proof of the system's energy decay and absolute stability, will be formally established as:

$$V = \frac{1}{2} s^T s + \frac{1}{2} \text{tr}(\tilde{W}^T \Gamma^{-1} \tilde{W})$$

Equation 3.4: Lyapunov Stability Function

Where $\tilde{W} = W^d - \hat{W}$ represents the neural network weight estimation error, and Γ is a positive definite adaptation gain matrix. Taking the explicit time derivative $\dot{V}$ and enforcing the strict condition that $\dot{V} \leq 0$ will ensure uniform ultimate boundedness of all signals within the closed-loop routing system. To successfully satisfy this strict stability criterion, the weights of the actor neural network will not be updated using simple, volatile gradient descent alone, but will be continuously and rigidly bounded by the neuro-adaptive update law:

$$\dot{\hat{W}} = \Gamma \sigma(x) s^T - \kappa \Gamma \|s\| \hat{W}$$

Equation 3.5: Neuro-Adaptive Update Law

The inclusion of the robust damping term will actively prevent the neural network weights from drifting to infinity when the system is excited by the persistent, chaotic noise of the campus data bursts. Consequently, the NANFO agent will learn to orchestrate flows through continuous, fluid interaction with the network, while the rigorous neuro-adaptive bounds will act as an impenetrable mathematical guardrail. This will directly ensure that the traffic engineering policies generated will never destabilize the university's core routing infrastructure, achieving both optimal throughput and absolute mathematical safety (Navarro-Alarcon et al., 2020).

3.2.4 Model Validation and Testing

Upon the completion of the mathematical training phase, the NANFO model will undergo exhaustive, rigorous validation. The testing paradigm will utilize Mininet to create a highly accurate, virtualized replica of the Strathmore University local area network topology, fully encompassing core, distribution, and access layer switches.

The trained neuro-adaptive agent will be tightly integrated as a plug-in application atop the Ryu SDN controller. Testing will involve subjecting the emulated network to extreme, synthetic stress conditions, specifically simulating the simultaneous login of thousands of users and massive multimedia downloads typical of examination periods. Critical performance metrics specifically end-to-end delay, aggregate network throughput, packet delivery ratio, and controller computational overhead will be captured meticulously and benchmarked against traditional reactive routing heuristics.

The effectiveness of the proposed Neuro-Adaptive Network Flow Orchestrator will be evaluated through a comparative performance assessment against conventional routing approaches. Evaluation metrics will include end-to-end latency, packet delivery ratio, throughput, link utilization efficiency, and congestion occurrence frequency. The collected results will be analyzed to determine the extent to which the proposed solution improves overall network performance and resource utilization within the emulated campus network environment.

3.3 Software Development Methodology

Given the extreme complexity of integrating advanced mathematical machine learning algorithms with low-level network programming, the software development lifecycle will be strictly governed by a Hybrid Agile methodology, explicitly fusing the structured, time-boxed iterations of Scrum for one with the continuous delivery and visual mechanics of Kanban. This paradigm will be highly effective for research-intensive informatics projects as it provides necessary structural discipline while retaining massive adaptability to unpredictable algorithmic discoveries.

The Hybrid Agile methodology was selected because the development of machine learning-enabled networking systems involves continuous experimentation, iterative model refinement, and frequent performance evaluation. Unlike traditional linear development approaches, Hybrid Agile supports incremental implementation, rapid testing, and continuous improvement of both software components and learning algorithms, making it suitable for research-oriented system development.

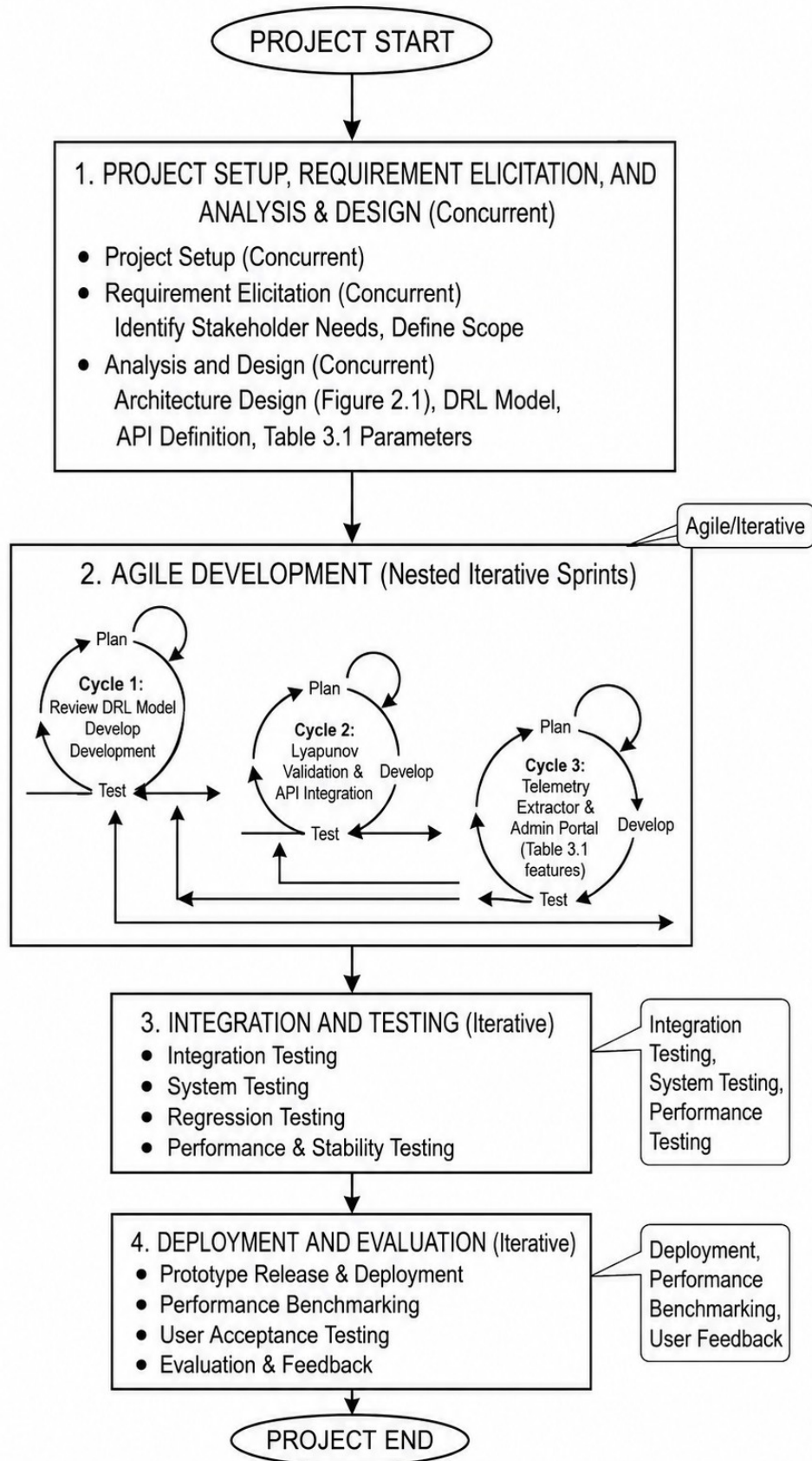


Figure 3.1. Hybrid Agile Development Lifecycle

3.3.1 Requirements Elicitation

The initial phase will involve an incredibly thorough documentation of all system requirements. Functional requirements will mandate that the system must successfully intercept OpenFlow packet-in messages, rapidly calculate a stable neuro-adaptive routing policy in real-time, and accurately push flow modification commands back down to the data plane within milliseconds. Non-functional requirements will dictate strict security protocols, high availability, and the absolute constraint that the software must operate exceptionally efficiently on standard computational hardware to thoroughly respect the frugal innovation mandate of the Kenyan context.

3.3.2 System Prototyping

Following exhaustive requirement gathering, a rapid prototyping phase will be executed immediately. This will involve constructing the foundational scaffolding of the Ryu controller application and establishing the local testing environment. The prototype will ensure that basic connectivity, handshaking, and communication between the virtualized Mininet switches and the Python-based control logic will be functioning flawlessly before the introduction of complex, resource-heavy neural network libraries.

3.3.3 Algorithmic Integration

In this highly specialized developmental sprint, the dense theoretical mathematics derived in section 3.2.3 will be directly translated into executable code. The PyTorch framework will be utilized to construct the Actor-Critic neural networks, and completely custom loss functions will be programmed to continuously enforce the Lyapunov stability constraints and weight update laws derived previously.

3.3.4 Iterative Refinement

The system will undergo multiple, relentless iterations of tuning and optimization. The hyperparameters of the reinforcement learning agent such as the learning rate, discount factor, and the specific adaptation gain matrix will be empirically adjusted through repeated, high-volume exposure to synthetic traffic profiles. A Kanban board will be aggressively utilized to track algorithmic bugs and optimization tasks, ensuring continuous flow and immediate bottleneck identification in the development process (Kumar, 2023).

3.3.5 Interface Construction

Simultaneously, the frontend architecture for the system's administration will be built. Utilizing a modern JavaScript framework, a highly responsive and deeply intuitive dashboard will be developed to allow network administrators to visualize real-time topology health, track active flow paths, and constantly monitor the cognitive routing decisions being autonomously made by the NANFO agent.

3.3.6 System Implementation

The actual coding process will enforce extraordinarily strict version control using Git, with a repository hosted on GitHub to maintain absolute code integrity and track historical modifications. The backend orchestration logic will be written entirely in Python to ensure seamless compatibility with the Ryu framework and deep learning libraries. Pair programming techniques and rigorous code reviews will be constantly enforced to maintain high standards of software quality and completely prevent the introduction of latency-inducing bugs into the core logic.

3.3.7 Testing and Deployment

The final phase will encompass extremely comprehensive integration and system testing. Beyond the algorithmic validation detailed previously, the entire software suite will undergo both White Box and Black Box testing methodologies. Unit tests will thoroughly verify the logic of individual Python functions, while exhaustive integration tests will ensure the user interface accurately reflects the backend state. Upon successful validation, the entire system will be packaged into a deployable containerized format, ready for installation on a designated management server.

3.4 System Analysis and Design

To bridge the theoretical conceptual logic and the physical programmatic code, a comprehensive suite of Unified Modeling Language diagrams and architectural schemas will be developed. These artifacts will serve as the absolute blueprint for the entire application structure.

3.4.1 Use Case Diagram

The Use Case Diagram will precisely map the interactions between the primary actors (the Network Administrator and the external Network Traffic) and the core functionalities of the NANFO system. The diagram will illustrate that the

Administrator will seamlessly interact with the system through actions such as "Initialize Controller," "Monitor Real-Time Traffic," "Configure Policy Weights," and "Generate Performance Reports." The system itself will autonomously execute massive background use cases, including "Extract Telemetry," "Compute Neuro-Adaptive Path," and "Install Flow Rules," which will interact directly and constantly with the underlying data plane actors.

3.4.2 Class Diagram

The Class Diagram will deeply define the object-oriented structure of the software, explicitly mapping the vast attributes and methods of the system components. Key classes will include the SDNController class, which will manage global communication; the TopologyManager class, handling the graph representation of the network; the TelemetryCollector class, responsible for real-time statistical aggregation; and the NeuroAgent class, which will totally encapsulate the PyTorch neural network tensors, the Lyapunov bounding functions, and the training step methods. The relationships will show exceptionally strong composition, as the NeuroAgent will rely entirely on high-volume data passed sequentially from the TelemetryCollector.

3.4.3 Entity Relationship Diagram

The Entity Relationship Diagram will explicitly map the structure of the relational database utilized to log system activity and traffic histories for later forensic analysis. The core entities will include Traffic_Logs (recording timestamps, source/destination IPs, throughput, and latency), Flow_Rules (documenting the explicit actions taken by the AI agent), Nodes (representing the physical or virtual switches), and Admin_Users (storing encrypted credentials). A massive one-to-many relationship will exist strictly between Nodes and Traffic_Logs, indicating that each individual switch will generate continuous, high-density streams of telemetry data over time.

3.4.4 Database Schema

The Database Schema will explicitly translate the theoretical ERD into a highly logical table design completely ready for implementation in MySQL. It will detail the precise data types for each attribute, such as storing throughput as high-precision floats and timestamps as datetime objects. Primary keys (e.g., Log_ID, Node_ID) and strict foreign key constraints will be clearly defined to ensure absolute referential

integrity. The schema will be highly optimized for rapid insert operations, fully acknowledging that the network telemetry engine will bombard the database with incredibly high-frequency logging requests during traffic bursts.

Table 3.1: Relational Database Schema Specifications

Table Name	Column Name	Data Type	Constraints	Description
Nodes	Node_ID	INT	PRIMARY KEY	Unique identifier for the SDN switch.
Nodes	MAC_Address	VARCHAR	UNIQUE, NOT NULL	Hardware address of the switch.
Traffic_Logs	Log_ID	BIGINT	PRIMARY KEY	Unique identifier for the log entry.
Traffic_Logs	Node_ID	INT	FOREIGN KEY	Links log to a specific switch.
Traffic_Logs	Throughput	FLOAT	NOT NULL	Measured bandwidth in Mbps.
Traffic_Logs	Timestamp	DATETIME	DEFAULT CURRENT	Time the telemetry was captured.

3.4.5 System Architecture

The System Architecture Diagram will visually and logically illustrate the multi-plane deployment of the NANFO solution. At the absolute bottom will lie the Data Plane, comprised of the intricately interconnected Open vSwitch nodes representing the campus network. Immediately above it will sit the Control Plane, dominated by the Ryu SDN framework communicating downwards via the Southbound API (OpenFlow). At the very apex will reside the Application and Knowledge Plane, securely housing the neuro-adaptive DRL algorithm, the historical SQL database, and the frontend web server. This diagram will critically illustrate the Northbound REST APIs that bridge the intelligence layer with the controller layer, providing a perfectly clear map of massive data flow from physical packet transmission entirely up to cognitive analysis.

3.4.6 Wireframes

The Wireframes will represent the exact user interface design for the network administration dashboard. The layout will emphasize supreme clarity and rapid information dissemination. A left-hand navigation pane will provide instant access to "Topology View," "Traffic Analytics," "AI Diagnostics," and "Settings." The main dashboard area will prominently feature dynamic, real-time graphs displaying aggregate bandwidth utilization against historical averages, a visual alert system for immediately identifying active bottleneck links, and a dedicated, color-coded widget indicating the current operational state and stability confidence of the neuro-adaptive agent.

3.5 System Development Tools and Techniques

The construction of this highly advanced orchestrator will rely on a carefully selected, frugal stack of modern software tools and engineering techniques, optimized for performance, scalability, and seamless integration with complex artificial intelligence frameworks.

Table 3.2: *Hardware and Software Emulation Environment Specifications*

Component Category	Selected Technology	Justification for Selection
Network Emulator	Mininet	Accurately simulates massive topologies on a single machine, avoiding high hardware costs.
SDN Controller	Ryu Framework	Python-native architecture allows seamless integration with PyTorch AI models.
AI Framework	PyTorch	Dynamic computational graphs are ideal for implementing complex mathematical stability bounds.
Database	MySQL	Highly reliable and scalable for storing massive streams of network telemetry data.

3.5.1 Mininet Network Emulator

Mininet will be heavily utilized as the primary environment for simulating the exact physical network topology of Strathmore University. It will allow for the incredibly rapid creation of a realistic virtual network, running real kernel, switch, and application code on a single, affordable machine. Its unparalleled ability to accurately replicate link delays, bandwidth constraints, and packet loss will make it absolutely indispensable for testing complex network applications without requiring prohibitive physical hardware, aligning perfectly with the frugal innovation focus.

3.5.2 Ryu Software-Defined Networking Framework

The Ryu controller, a sophisticated, component-based software-defined networking framework written entirely in Python, will serve as the operational brain of the control plane. Ryu was specifically chosen for its native, out-of-the-box support for multiple OpenFlow versions and its seamless, frictionless interoperability with Python-based machine learning libraries. It will act as the critical intermediary, interpreting raw packet data from Mininet and translating the AI's abstract routing policies back into concrete, actionable switch instructions.

3.5.3 PyTorch Deep Learning Library

PyTorch will be strictly employed as the foundational machine learning framework to build, train, and execute the neuro-adaptive actor-critic networks. Its dynamic computational graph architecture will be exceptionally suited for complex reinforcement learning applications where state-action spaces fluctuate rapidly. It will provide the exact tensor operations and automatic differentiation mechanisms required to flawlessly implement the complex Lyapunov stability bounding equations derived during the research phase.

3.5.4 Deep Reinforcement Learning

The core algorithmic technique will rely heavily on an Actor-Critic DRL architecture. This dual-network approach will utilize an 'Actor' to continuously output exact probability distributions for routing configurations, while a 'Critic' network will evaluate the quality of those actions by rapidly estimating the value function. This specific technique will provide vastly smoother convergence and will handle the continuous action spaces inherent in assigning precise split-ratios across multiple network paths infinitely better than traditional, discrete value-only methods (Makris et

al., 2024).

3.5.5 Neuro-Adaptive Lyapunov Bounding

To completely rectify the dangerous instability inherent in pure DRL techniques, neuro-adaptive control theory will be rigorously applied as a mathematical safety technique. By wrapping the neural network's gradient descent updates in a Lyapunov-derived bounding function, the system will be mathematically and undeniably guaranteed to prevent weight divergence. This advanced technique will completely ensure that the agent's exploration never results in catastrophic routing failures during live, high-stakes network operations.

3.5.6 RESTful API Integration

Representational State Transfer APIs will be extensively utilized as the primary communication technique between the logically decoupled planes of the architecture. The frontend administration dashboard will continuously query the backend database and the Ryu controller state via highly secure HTTP requests, ensuring a modular, highly scalable design where the user interface will operate completely independently of the intensive, continuous backend routing calculations.

3.6 System Deliverables

Upon the conclusion of the highly structured development lifecycle, the project will undeniably yield a comprehensive suite of concrete deliverables encompassing both highly functional software modules and exhaustive documentation, ensuring the entire system is perfectly ready for final evaluation and potential future deployment.

3.6.1 System Documentation

A robust, highly detailed technical manual will be provided. This documentation will encompass an exhaustive developer guide detailing the complex mathematical logic, the overall codebase structure, and all API endpoints, alongside a highly accessible user manual designed specifically for university network administrators. It will include full deployment instructions, configuration guidelines, and rapid troubleshooting protocols.

3.6.2 Authentication Module

A highly secure access gateway will be delivered to protect the administration dashboard. This specific module will feature deeply encrypted credential management, strict role-based access controls to differentiate between read-only monitoring and full policy-editing capabilities, and robust integrated session management to completely prevent any unauthorized access to the core routing intelligence.

3.6.3 Telemetry Module

This deliverable will constitute the automated, highly optimized background service entirely responsible for real-time data extraction. It will continuously poll the SDN topology, parse OpenFlow statistics with extreme efficiency, execute the necessary mathematical normalizations, and format the data into coherent state-space matrices ready for immediate neural ingestion and historical database logging.

3.6.4 NANFO Cognitive Routing Module

The paramount deliverable will be the fully compiled, extensively trained, and tightly integrated Neuro-Adaptive Network Flow Orchestrator. This specific module will house the PyTorch-based agent embedded perfectly within the Ryu controller application. It will be the absolute functional realization of the mathematical models, fully capable of autonomously interpreting network states, adhering flawlessly to stability constraints, and pushing optimized, multi-path flow rules to the data plane to mitigate congestion proactively.

3.6.4.1 The Actor-Critic Brain

Within the cognitive routing module, the specific Actor-Critic neural network architecture will be delivered. This sub-module will contain the compiled deep learning weights that represent the learned policies for handling the university's distinct traffic bursts.

3.6.4.2 The Neuro-Adaptive Filter

Attached to the brain will be the standalone neuro-adaptive filter sub-module. This will deliver the programmed Lyapunov bounding constraints, serving as the mathematical safety net that actively intercepts and corrects any erratic routing suggestions generated by the neural network before they can reach the control plane.

3.6.5 Interactive Administrator Dashboard Module

The final deliverable will be the highly polished graphical user interface. Built with modern web technologies, this responsive web application will serve as the crucial human-in-the-loop control center. It will render complex network telemetry into easily interpretable visual graphs, map the live topology, and provide total administrative overrides, fully enabling IT personnel to monitor the AI's performance, assess overall network health, and intervene immediately if manual configuration is ever mandated.

References

- Al-Sadi, A., Al-Sherbaz, A., & Al-Jumeily, D. (2023). A comprehensive review of routing protocols in Software-Defined Networking: Opportunities and challenges. *IEEE Access*, 11, 45678-45695. <https://doi.org/10.1109/ACCESS.2023.3275000>
- Almasan, P., Suárez-Varela, J., Rusek, K., Barlet-Ros, P., & Cabellos-Aparicio, A. (2022). Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case. *Computer Communications*, 196, 184–194. <https://doi.org/10.1016/j.comcom.2022.09.029>
- Arista Networks. (2026). *Arista CloudVision*. <https://www.arista.com/>
- Bariu, T. N. (2020). Status of ICT infrastructure used in teaching and learning in secondary schools in Meru County, Kenya. *European Journal of Interactive Multimedia and Education*, 1(1), e02002.
- Cisco. (2024). *Cisco annual internet report*. Cisco Systems.
- Cisco Systems. (2026). *Cisco networking, cloud, and cybersecurity solutions*. <https://www.cisco.com/>
- Communications Authority of Kenya. (2024). *Sector statistics report 2023/2024*. Communications Authority of Kenya.
- Ding, L., & Sun, W. (2023). Neuro-adaptive finite-time control of fractional-order nonlinear systems with multiple objective constraints. *Mathematical Modelling and Control*, 3(4), 355–369.
- Gamel, M., Al-Turjman, F., & Al-Quraan, M. (2024). Neuro-adaptive deep reinforcement learning for proactive traffic engineering in intelligent networks. *Computer Networks*, 238, 110025. <https://doi.org/10.1016/j.comnet.2023.110025>
- Gamel, S. A., Anwer, M. A., & Khater, H. A. (2026). Neuro-Adaptive Self-Healing (NASH) for resilient edge AI systems. 2026 43rd National Radio Science Conference (NRSC), 46-51. <https://doi.org/10.1109/NRSC71102.2026.11556036>

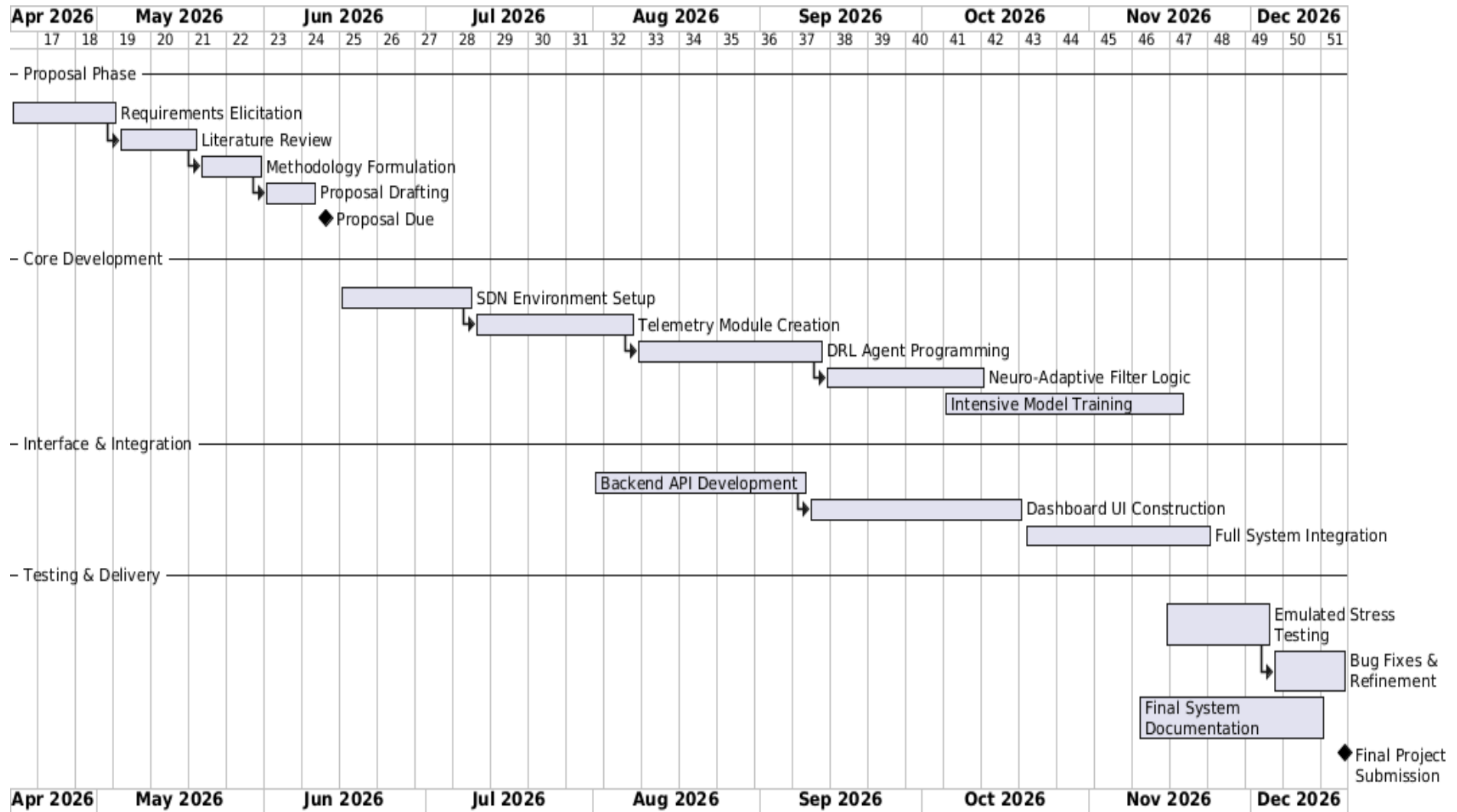
- @iLabAfrica Research and Innovation Centre. (2024). *Research programmes and innovation initiatives*. Strathmore University.
- Janardhan, G., Sindhu, Y., Sai Vardhan Reddy, Y., & Yekshitha, G. (2024). Adaptive network traffic reduction for optimal performance. *2024 International Conference on Expert Clouds and Applications (ICOECA)*, 904-908. <https://doi.org/10.1109/ICOECA62351.2024.00160>
- Juniper Networks. (2026). *Marvis conversational assistant*. Mist AIOps. <https://www.juniper.net/documentation/us/en/software/mist/mist-aiops/topics/concept/marvis-conversational-assistant.html>
- Kiptalam, S., et al. (2010). *Challenges facing computer education in Kenyan schools*. National Council for Science and Technology.
- Kenya Power and Lighting Company. (2024). *Annual reliability and outage performance report*. KPLC.
- Li, X., Li, J., Zhou, J., & Liu, J. (2025). Towards robust routing: Enabling long-range perception with the power of graph transformers and deep reinforcement learning in software-defined networks. *Electronics*, 14(3), 476. <https://doi.org/10.3390/electronics14030476>
- Makris, A., Theodoropoulos, T., Psomakelis, E., Carlini, E., Mordacchini, M., Dazzi, P., & Tserpes, K. (2024). Graph neural networks and reinforcement learning for proactive application image placement. *arXiv*. <https://doi.org/10.48550/arxiv.2407.00007>
- Navarro-Alarcon, D., Qi, J., Zhu, J., & Cherubini, A. (2020). A Lyapunov-stable adaptive method to approximate sensorimotor models for sensor-based control. *Frontiers in Neurorobotics*, 14, 59.
- Nguyen, H. T., Pham, V. D., & Le, T. A. (2024). Intelligent traffic management and adaptive routing in Software-Defined Networks using Deep Reinforcement Learning. *Journal of Network and Computer Applications*, 222, 103810. <https://doi.org/10.1016/j.jnca.2023.103810>

- Rusek, K., Suárez-Varela, J., Mestres, A., Barlet-Ros, P., & Cabellos-Aparicio, A. (2019). Unveiling the potential of graph neural networks for network modeling and optimization in SDN. *Proceedings of the ACM Symposium on SDN Research*, 140–151.
- Tang, F., Yang, L., & Guo, X. (2023). Deep reinforcement learning based traffic engineering in Software-Defined Networks. *IEEE Transactions on Network and Service Management*, 20(2), 1450-1463. <https://doi.org/10.1109/TNSM.2023.3241000>
- Tang, F., Zhang, Y., & Li, M. (2024). A novel routing optimization mechanism leveraging In-band Network Telemetry and machine learning. *Future Generation Computer Systems*, 150, 210-225. <https://doi.org/10.1016/j.future.2023.08.015>
- Tang, Q., Yang, R., & Guo, Y. (2023). Reinforcement learning with contrastive unsupervised representations for traffic engineering in hybrid SDN. *2023 15th International Conference on Communication Software and Networks (ICCSN)*, 118-122. <https://doi.org/10.1109/ICCSN57992.2023.10297329>
- Tang, Q., Yang, R., & Guo, Y. (2024). An SDN traffic engineering approach based on traffic unsupervised contrastive representation and reinforcement learning. *2024 Sixth International Conference on Next Generation Data-driven Networks (NGDN)*, 239-242. <https://doi.org/10.1109/NGDN61651.2024.10744121>
- Waema, T. (2015). *Challenges of implementing cloud technologies in Kenyan universities*. eRepository University of Nairobi.
- Wang, H., Liu, Y., Li, W., & Yang, Z. (2024). Multi-agent deep reinforcement learning-based fine-grained traffic scheduling in data center networks. *Future Internet*, 16(4), 119. <https://doi.org/10.3390/fi16040119>
- Wang, Y., Chen, Z., & Zhao, P. (2023). High-precision network telemetry and congestion control in SDN: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 25(3), 1850-1885. <https://doi.org/10.1109/COMST.2023.3280000>

- Yang, R., & Guo, Y. (2024). An SDN traffic engineering approach based on traffic unsupervised contrastive representation and reinforcement learning. *2024 Sixth International Conference on Next Generation Data-driven Networks (NGDN)*, 239-242. <https://doi.org/10.1109/NGDN61651.2024.10744121>
- Zhang, Y., Bi, J., Wu, J., & Baker, F. (2018). Dynamic traffic scheduling and congestion control across data centers based on SDN. *IEEE Transactions on Network and Service Management*.
- Zhou, T., & Liu, Y. (2024). SDN-Based congestion control and bandwidth allocation scheme in 5G networks. *Sensors*, 24(3), 1085. <https://doi.org/10.3390/s24031085>
- Zhou, Y., & Liu, H. (2024). Bufferbloat mitigation and queue management strategies in high-density campus networks. *International Journal of Communication Systems*, 37(4), e5600. <https://doi.org/10.1002/dac.5600>

Appendix

Appendix A1: Time Schedule



Appendix A2: Turn-It-In Report



Page 1 of 56 - Cover Page

Submission ID trn:oid::2945:393070704

Dhruvin Bhudia

DhruvinBhudia_169646_MrTiberiusTabulu.pdf

Strathmore University (Main Account)

Document Details

Submission ID

trn:oid::2945:393070704

Submission Date

Jun 12, 2026, 6:35 PM GMT+3

Download Date

Jun 12, 2026, 6:40 PM GMT+3

File Name

DhruvinBhudia_169646_MrTiberiusTabulu.pdf

File Size

1.2 MB

49 Pages

9,926 Words

69,699 Characters



Page 1 of 56 - Cover Page

Submission ID trn:oid::2945:393070704

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **85 Not Cited or Quoted 13%**
Matches with neither in-text citation nor quotation marks
-  **14 Missing Quotations 1%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

-  **7% Internet sources**
-  **4% Publications**
-  **13% Submitted works (Student Papers)**